■ Building Your Own Programming Language

A practical beginner-to-intermediate handbook for designing a language, building a lexer and parser, creating an AST, and executing programs with an interpreter.

Reference implementation: the custom interpreter project built in Python

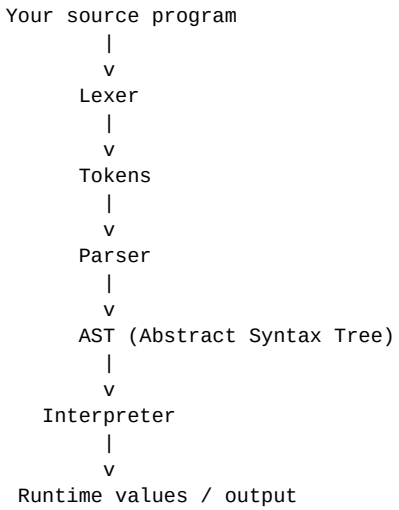
Author
Danish Khan

B.Tech CSE • Software Development • DSA • AI • Programming Language Design

This handbook explains the ideas behind the project so that another developer can follow the same path and create their own programming language.

1. What You Are Building

A programming language is a set of rules that lets humans express computations. An interpreter is a program that reads source code written in that language and executes it. The project documented here follows a classic small-interpreter architecture.



The reference implementation is written in Python, but the language itself is a separate language. That distinction is important: Python is the implementation language; the custom syntax is the language being interpreted.

What you will learn

• How to define keywords, operators, literals, and grammar rules.

• How a lexer converts characters into tokens.

• How a recursive-descent parser converts tokens into an AST.

• How an interpreter evaluates AST nodes.

• How runtime values, functions, scopes, and symbol tables work.

• How to design built-in functions and runtime errors.

• How to extend a language safely with new features.

Reference syntax at a glance

```
VAR x = 10
PRINT(x)

IF x > 5 THEN
    PRINT("large")
ELSE
    PRINT("small")
END

FOR i = 0 TO 5 STEP 1 THEN
    PRINT(i)
END

FUNC add(a, b) -> a + b
PRINT(add(2, 3))
```

The exact reference keywords in the implementation are VAR, AND, OR, NOT, IF, ELIF, ELSE, FOR, TO, STEP, WHILE, THEN, FUNC, END, RETURN, CONTINUE, and BREAK. The lexer also recognizes numbers, strings, identifiers, arithmetic/comparison operators, parentheses, square brackets, commas, arrows, and newlines.

2. The Interpreter Pipeline

2.1 Lexer

The lexer scans raw characters from left to right. It groups characters into meaningful tokens. For example, VAR x = 10 becomes a sequence containing a keyword token, an identifier token, an equals token, and an integer token.

```
VAR x = 10
```

```
KEYWORD(VAR)
IDENTIFIER(x)
EQ
INT(10)
NEWLINE
EOF
```

2.2 Parser

The parser consumes tokens according to grammar rules. The reference implementation uses recursive-descent style methods such as expr(), comp_expr(), arith_expr(), term(), factor(), call(), list_expr(), if_expr(), for_expr(), while_expr(), and func_def().

2.3 AST

An Abstract Syntax Tree stores the structure of a program. It is much easier to execute an AST than raw text because the parser has already decided what each construct means.

```
VAR x = 10 + 5
```

```
VarAssignNode
    |
    +-- variable: x
    |
    +-- value:
          BinOpNode
          /      \
        10        5
```

2.4 Interpreter

The interpreter visits AST nodes and produces runtime values. The implementation uses a visitor pattern: a node such as Numbernode is dispatched to visit_Numbernode(), while a function call is dispatched to visit_CallNode().

2.5 Runtime

The runtime contains values such as Number, String, List, Function, and BuiltInFunction. Contexts and SymbolTable objects provide variable storage and scope.

3. Designing Your Language Before Coding

Before writing the lexer, write down the language rules. Start small. A useful first version only needs:

• Numbers and strings

• Variables

• Arithmetic

• Comparisons

• One conditional

• Functions

• A simple REPL or run() entry point

Choose a style

Decide early how statements end, how blocks are delimited, and how variables/functions are declared. The reference language uses NEWLINE and END for blocks, with THEN after conditions.

```
IF condition THEN
    ...
END

WHILE condition THEN
    ...
END
```

Choose your tokens

The reference implementation defines token types for integers, floats, strings, plus/minus/multiply/divide/power, identifiers, keywords, assignment, equality/inequality, comparisons, commas, arrows, newlines, parentheses, square brackets, and EOF.

Grammar is your contract

A grammar is the contract between the lexer and parser. For example:

```
expression  := VAR assignment | logic_expression
logic       := comparison (AND | OR comparison)*
comparison  := arithmetic (== | != | < | > | <= | >= arithmetic)*
arithmetic  := term (+ | - term)*
term        := factor (* | / factor)*
factor      := (+ | - | NOT) factor | power
power       := call (^ factor)?
call        := atom ( "(" arguments? ")" )?
atom        := number | string | identifier | list | "(" expression ")"
```

This is a teaching-level grammar, not a replacement for the exact parser code. When you extend the language, update the lexer, grammar/parser, AST, interpreter, and tests together.

4. Building the Lexer

4.1 Characters → tokens

The lexer reads one character at a time. Whitespace is usually skipped; newlines become NEWLINE tokens; punctuation becomes operator tokens; identifiers are checked against the keyword list.

Numbers

```
10
3.14
0.5
```

The reference lexer recognizes integers and floating-point numbers by counting decimal points.

Strings

```
"Hello, world!"
"Line 1\nLine 2"
"He said: \"hello\""
```

The implementation supports escape characters including newline, tab, quotation marks, and backslash.

Comments

```
# This is a comment
VAR x = 10  # Inline comment
```

The reference lexer skips text beginning with # until the end of the line.

Operators

```
+   -   *   /   ^
=   ==  !=  <   >   <=  >=
->
```

Why tokens matter

If the lexer produces bad tokens, the parser cannot recover reliably. A good lexer is predictable, reports illegal characters clearly, and preserves source positions for error messages.

5. Building the Parser

5.1 Recursive descent

A recursive-descent parser gives each grammar rule a Python method. A parser method consumes tokens and returns a node plus an optional error.

```
def expr(self):
    ...
    return res.success(node)
```

Operator precedence

The reference parser separates expression levels so multiplication binds more tightly than addition, and comparisons happen after arithmetic.

```
2 + 3 * 4
```

This should mean 2 + (3 * 4), not (2 + 3) * 4.

Parentheses

```
(2 + 3) * 4
```

Parentheses override the normal precedence.

Function calls

```
PRINT("Hello")
add(2, 3)
LEN([1, 2, 3])
```

Lists

```
[1, 2, 3]
[]
["Danish", 10, TRUE]
```

Parser errors

Every expected token should have a useful error message. For example, after FUNC the parser expects a parameter list in parentheses. After a conditional expression it expects THEN. A block must eventually reach END.

6. Statements and Control Flow

6.1 Variables

```
VAR x = 10
VAR y = x + 5
PRINT(y)
```

The parser creates a variable-assignment node. The interpreter evaluates the right-hand side and stores the resulting runtime value in the current symbol table.

6.2 IF / ELIF / ELSE

```
VAR score = 82

IF score >= 90 THEN
    PRINT("A")
ELIF score >= 75 THEN
    PRINT("B")
ELSE
    PRINT("C")
END
```

6.3 WHILE

```
VAR x = 0

WHILE x < 5 THEN
    PRINT(x)
    VAR x = x + 1
END
```

When implementing assignment semantics, decide whether VAR redeclares a name or updates an existing name. Document that decision because it affects examples and scope.

6.4 FOR

```
FOR i = 0 TO 5 THEN
    PRINT(i)
END
```

6.5 STEP

```
FOR i = 0 TO 10 STEP 2 THEN
    PRINT(i)
END
```

The reference runtime chooses the loop direction from the step value and repeatedly assigns the loop variable.

6.6 BREAK and CONTINUE

```
FOR i = 0 TO 10 THEN
    IF i == 5 THEN
        BREAK
    END
    PRINT(i)
END

FOR i = 0 TO 5 THEN
    IF i == 2 THEN
        CONTINUE
    END
    PRINT(i)
END
```

7. Functions and Scope

7.1 Expression-returning functions

```
FUNC add(a, b) -> a + b

PRINT(add(10, 20))
```

7.2 Multiline functions

```
FUNC square(x)
    RETURN x * x
END

PRINT(square(6))
```

7.3 Return

```
FUNC max(a, b)
    IF a > b THEN
        RETURN a
    ELSE
        RETURN b
    END
END
```

7.4 Function calls

A call has a target and zero or more argument nodes. At runtime, the interpreter evaluates the arguments, creates a function execution context, binds argument names to argument values, and executes the function body.

```
FUNC greet(name) -> PRINT_RET(name)

greet("Danish")
```

7.5 Symbol tables

```
Global Symbol Table
    |
    +-- x
    +-- PRINT
    +-- TRUE
    |
    +-- Function Context
           |
           +-- a
           +-- b
```

The parent pointer in SymbolTable lets a child scope look up names in its parent scope. This is the foundation of lexical/runtime scope.

8. Runtime Values

Numbers

```
VAR a = 10
VAR b = 2.5

PRINT(a + b)
PRINT(a > b)
```

Strings

```
VAR name = "Danish"
PRINT(name)
PRINT("Hello\nWorld")
```

Lists

```
VAR nums = [10, 20, 30]
PRINT(nums)
```

Operations

A runtime value should expose only operations that make sense for its type. For example, numbers can support arithmetic; lists can support append/pop/extend; functions can support execution.

Runtime errors

Runtime errors should include source positions and the current execution context whenever possible. Examples include undefined variables, invalid argument types, division by zero, and out-of-range list indices.

```
PRINT(unknown_variable)
```

A useful error should identify what failed, where it failed, and enough context for the programmer to fix it.

9. Built-in Functions

Built-ins are functions supplied by the interpreter instead of being written in the custom language.

Output

```
PRINT("Hello")
PRINT(123)
```

Input

```
VAR name = INPUT()
PRINT(name)

VAR age = INPUT_INT()
PRINT(age)
```

Type checks

```
IS_NUM(10)
IS_STR("hello")
IS_LIST([1, 2])
IS_FUNC(PRINT)
```

List operations

```
VAR nums = [1, 2, 3]

APPEND(nums, 4)
PRINT(nums)

POP(nums, 0)
PRINT(nums)

EXTEND(nums, [5, 6])
PRINT(nums)

PRINT(LEN(nums))
```

RUN

```
RUN("another.basic")
```

RUN loads another source file and sends it through the same run() pipeline. A production implementation should add clear file-not-found and execution-error reporting.

10. Complete Example Programs

Example 1 — Calculator

```
VAR a = 20
VAR b = 5

PRINT(a + b)
PRINT(a - b)
PRINT(a * b)
PRINT(a / b)
```

Example 2 — Even/Odd

```
VAR n = 12

IF n / 2 * 2 == n THEN
    PRINT("Even")
ELSE
    PRINT("Odd")
END
```

This example demonstrates the language structure. If you later add a modulo operator, replace the arithmetic test with n % 2 == 0.

Example 3 — Function

```
FUNC factorial(n)
    IF n <= 1 THEN
        RETURN 1
    END
    RETURN n * factorial(n - 1)
END

PRINT(factorial(5))
```

Example 4 — List processing

```
VAR values = [1, 2, 3]

APPEND(values, 4)
APPEND(values, 5)

PRINT(LEN(values))
PRINT(values)
```

Example 5 — Loop with control flow

```
FOR i = 0 TO 10 THEN
    IF i == 3 THEN
        CONTINUE
    END

    IF i == 8 THEN
        BREAK
    END

    PRINT(i)
END
```

11. How to Create Your Own Language From Scratch

Phase 1 — Define the language

• Choose a name.

• Write 10–20 example programs before implementation.

• Choose keywords and punctuation.

• Define variables, types, operators, blocks, and functions.

• Write a small grammar.

Phase 2 — Build the lexer

• Create token constants.

• Create a Position object for line/column tracking.

• Scan whitespace and newlines.

• Scan numbers and strings.

• Scan identifiers and keywords.

• Scan operators and punctuation.

• Return an EOF token.

• Test every token independently.

Phase 3 — Build the parser

• Create AST node classes.

• Implement atoms first.

• Implement unary and binary operators.

• Implement assignment.

• Implement lists and calls.

• Implement conditionals and loops.

• Implement functions and return.

• Add useful syntax errors.

Phase 4 — Build the runtime

• Implement Number, String, List, and Function values.

• Create Context and SymbolTable.

• Implement the Interpreter visitor.

• Implement built-in functions.

• Add runtime error handling.

• Create a global symbol table.

Phase 5 — Test everything

```
# Lexer tests
VAR x = 10
```

```
x + 5

# Parser tests
IF x > 5 THEN
    PRINT(x)
END

# Runtime tests
FUNC add(a, b) -> a + b
PRINT(add(2, 3))
```

Test features in isolation before combining them. When a feature fails, identify whether the failure is in lexing, parsing, AST construction, or runtime execution.

12. Recommended Project Structure

```
my-language/
■
■■■ basic.py
■■■ string_with_arrows.py
■■■ README.md
■■■ HANDBOOK.pdf
■
■■■ examples/
■   ■■■ hello.basic
■   ■■■ variables.basic
■   ■■■ conditions.basic
■   ■■■ loops.basic
■   ■■■ functions.basic
■   ■■■ lists.basic
■
■■■ tests/
    ■■■ test_lexer.py
    ■■■ test_parser.py
    ■■■ test_runtime.py
```

The reference project currently concentrates the interpreter implementation in basic.py and uses a string-with-arrows helper for source-position error rendering. As the language grows, splitting lexer, parser, AST, runtime values, and built-ins into modules will make the project easier to maintain.

Suggested future modules

```
lexer.py
tokens.py
parser.py
nodes.py
values.py
runtime.py
builtins.py
errors.py
main.py
```

13. Debugging and Common Mistakes

Undefined AST attributes

If the parser stores body_node but the interpreter reads body_value_node, execution fails. Use consistent names across node constructors and visitors.

Wrong argument counts

If a node constructor expects six arguments, every parser path that creates the node must supply six. Prefer keyword arguments for complicated nodes.

Wrong isinstance usage

```
# Wrong
isinstance(list_a, list_b)

# Correct
isinstance(list_a, List)
```

Scope bugs

If SymbolTable receives a parent but discards it, functions cannot see names from parent scopes. Always preserve the parent reference when constructing a nested scope.

Parser precedence bugs

When adding a new operator, decide its precedence and associativity before adding it to the parser. Add examples such as 2 + 3 * 4 and (2 + 3) * 4 to your tests.

Infinite loops

Input and loop implementations must have an exit path. A successful INPUT_INT conversion should return; invalid input should ask again.

A practical debugging method

```
1. Print the source.
2. Print the token stream.
3. Inspect the AST.
4. Run one AST node manually.
5. Inspect the current symbol table.
6. Inspect the runtime value.
7. Add a regression test.
```

14. How to Add a New Feature

Example: Adding a modulo operator

Suppose you want the language to support %.

Step 1 — Add a token

```
TT_MOD = 'MOD'
```

Step 2 — Teach the lexer

```
elif self.current_char == '%':
    tokens.append(Token(TT_MOD, pos_start=self.pos))
    self.advance()
```

Step 3 — Update the parser

Decide whether % belongs at the same precedence level as multiplication and division.

```
term := factor ((* | / | %) factor)*
```

Step 4 — Add an AST/runtime operation

```
elif node.op_tok.type == TT_MOD:
    result = left.mod(other)
```

Step 5 — Add tests

```
PRINT(10 % 3)
# expected: 1
```

This five-step pattern generalizes to almost every language feature: tokenization → parsing → AST → runtime → tests.

15. Testing Checklist

Area	Tests to write
Lexer	Numbers, floats, strings, escapes, comments, keywords, operators, EOF
Parser	Precedence, parentheses, variables, lists, calls, blocks, functions
Runtime	Arithmetic, comparisons, boolean logic, variables, lists
Control flow	IF, ELIF, ELSE, FOR, STEP, WHILE, BREAK, CONTINUE
Functions	Arguments, return, recursion, local scope, parent scope
Built-ins	PRINT, INPUT, type checks, list functions, RUN
Errors	Illegal characters, invalid syntax, undefined names, bad arguments, bounds

Minimum regression suite

```
VAR x = 10
PRINT(x + 5)

IF x > 5 THEN
    PRINT("yes")
ELSE
    PRINT("no")
END

FOR i = 0 TO 3 THEN
    PRINT(i)
END

FUNC add(a, b) -> a + b
PRINT(add(2, 3))

VAR xs = [1, 2]
APPEND(xs, 3)
PRINT(LEN(xs))
```

16. Publishing the Language on GitHub

A good repository should make it possible for a stranger to understand the language without asking you questions.

Recommended repository contents

```
README.md
HANDBOOK.pdf
basic.py
string_with_arrows.py
examples/
tests/
LICENSE
```

README should contain

• What the language is.

• Why you built it.

• Architecture diagram.

• Installation/run instructions.

• Syntax examples.

• Built-in functions.

• Roadmap.

• Contribution instructions.

Useful Git commands

```
git add .
git commit -m "Add programming language handbook"
git push origin main
```

Keep the handbook versioned with the language. Whenever syntax changes, update the examples and documentation at the same time.

17. Roadmap: From Small Interpreter to Real Language

Level 1 — Educational interpreter

```
Numbers
Strings
Variables
Arithmetic
IF
Loops
Functions
Lists
Built-ins
Errors
```

Level 2 — Better language

• Modulo operator

• Boolean type

• Indexing and assignment

• String concatenation

• Dictionaries/maps

• More standard-library functions

• Better diagnostics

• Automated tests

Level 3 — Developer experience

• REPL with history

• Syntax highlighting

• VS Code extension

• Formatter

• Language server

• Debugger

• Package/module system

Level 4 — Advanced implementation

• Bytecode compiler

• Virtual machine

• Optimizations

• Garbage collection

• Static analysis

• Optional type system

18. Final Takeaway

Creating a programming language is one of the best ways to understand what normally happens behind a compiler or interpreter. You are forced to think about syntax, grammar, data structures, scope, execution, errors, and language design as one connected system.

```
Characters
   ↓
Tokens
   ↓
AST
   ↓
Runtime
   ↓
Program behavior
```

Start with a tiny language. Make it work. Test it. Document it. Then add one feature at a time. The most important lesson is not the number of keywords your language has; it is understanding the complete path from source text to execution.

A final challenge

After understanding this handbook, try creating your own language instead of copying the reference syntax. Change at least three things: the keywords, the block syntax, and one core data type. Then implement the same lexer → parser → AST → interpreter pipeline.

```
YourLanguage Source
        ↓
     Lexer
        ↓
     Tokens
        ↓
     Parser
        ↓
      AST
        ↓
   Interpreter
        ↓
     Output
```

Project reference note: This handbook is based on the custom interpreter implementation and language syntax supplied for this project. It intentionally explains the architecture and examples rather than reproducing the entire implementation source code.

Built for learning • Building from scratch • Keep experimenting ■